

MODULE 03

SYLLABUS: Serial Communication: Serial Data Communication between Serial Monitor and Arduino, Software Serial. I2C: I2C Hardware Design, Communication Scheme and ID Numbers, Hardware Requirements and Pull-Up Resistors, Communicating with an I2C Temperature Probe. SPI: SPI Hardware Configuration, Communication Scheme, Comparing SPI to I2C and UART, Communicating with an SPI Accelerometer.

TO WATCH VIDEO CLASSES
OF THIS COURSE: [TAP HERE OR
SCAN QR CODE](#)



SERIAL COMMUNICATION

Serial communication is the process of transmitting data **one bit at a time** over a communication channel between two devices. It is the most fundamental way the Arduino communicates with a computer or with other devices.

The Arduino's most important serial capability is its **USB-to-serial interface**, which allows the Arduino to be programmed over USB and to exchange data with a host computer — all through a single cable, without any external programmer hardware.

The Arduino's Serial Hardware

The ATmega328P on the Arduino Uno has **one hardware USART** (Universal Synchronous/Asynchronous Receiver/Transmitter) port, accessible via:

- **Pin 0 (RX)** — Receive
- **Pin 1 (TX)** — Transmit

These pins are **multiplexed** — they serve both as general digital I/O and as the hardware serial interface. When serial communication is active, pins 0 and 1 should not be used for other digital I/O purposes.

USB-to-Serial Conversion Methods

Since computers use USB while the ATmega uses UART serial, a conversion is needed. Three approaches are used across different Arduino boards:

Method	Board Example	How It Works
Dedicated FTDI / Silicon Labs IC	Arduino Nano, Adafruit METRO 328	External IC (e.g. CP2104) converts USB↔Serial
Secondary ATmega MCU (ATmega16U2)	Arduino Uno	Second MCU runs LUFA firmware; converts USB↔Serial; board identified as "Arduino" by OS
Built-in USB in main MCU (ATmega32U4)	Arduino Leonardo, Micro	Main MCU handles USB directly; no secondary chip needed; can emulate keyboard/joystick

On the Arduino Uno, the ATmega16U2 secondary MCU converts USB signals to UART signals for the main ATmega328P. The Arduino IDE installs the necessary driver automatically on Windows.

Serial Communication Functions — Arduino Side

All serial communication is accessed through the Serial object. Before using any serial function, the port must be initialized in `setup()`.

Core Serial Functions

Function	Purpose
<code>Serial.begin(baud_rate)</code>	Initializes serial port at given baud rate (e.g. 9600)
<code>Serial.print(data)</code>	Sends data without a newline at the end
<code>Serial.println(data)</code>	Sends data followed by a newline character (<code>\n</code>)
<code>Serial.available()</code>	Returns number of bytes waiting in the incoming buffer
<code>Serial.read()</code>	Reads and removes one byte from the incoming buffer
<code>Serial.parseInt()</code>	Extracts and returns the next integer from the serial buffer
<code>Serial.setTimeout(ms)</code>	Sets timeout for <code>parseInt()</code> and similar functions

Baud Rate

The **baud rate** specifies the number of bits transferred per second. Both the Arduino sketch and the Serial Monitor must be set to the **same baud rate** — otherwise, garbled characters appear. The most common value used throughout this course is **9600 baud**.

Printing Data from Arduino to Serial Monitor

Basic Serial Print Program

```
const int POT = 0; // Potentiometer on analog pin A0

void setup() {
  Serial.begin(9600); // Initialize serial at 9600 baud
}

void loop() {
  int val = analogRead(POT); // Read ADC: 0–1023
  int per = map(val, 0, 1023, 0, 100); // Convert to percentage

  Serial.print("Analog Reading: ");
  Serial.print(val); // Print raw value (no newline)
  Serial.print(" Percentage: ");
  Serial.print(per); // Print percentage (no newline)
  Serial.println("%"); // Print % and newline
  delay(1000);
}
```

Using `Serial.print()` for intermediate values and `Serial.println()` only on the last item keeps all values on the same line in the monitor.

Special Characters for Formatting

Special characters are embedded in strings using a **backslash escape** sequence:

Escape Code	Character	Effect
\n	Newline	Moves cursor to next line
\t	Tab	Inserts horizontal tab spacing
\\n	Literal \n	Prints the characters backslash-n

Tabular Serial Output Program

```
const int POT = 0;

void setup() {
  Serial.begin(9600);
}

void loop() {
  Serial.println("\nAnalog Pin\tRaw Value\tPercentage");
  Serial.println("-----");

  for (int i = 0; i < 10; i++) {
    int val = analogRead(POT);
    int per = map(val, 0, 1023, 0, 100);

    Serial.print("A0\t"); // Two tabs for alignment
    Serial.print(val);
    Serial.print("\t");
    Serial.print(per);
    Serial.println("%");
    delay(1000);
  }
}
```

This produces neatly aligned columns in the Serial Monitor with tab-separated values.

Serial Data Format Options

Serial.print() and Serial.println() accept an optional second argument specifying the number format:

Second Argument	Format	Example (value=23)	Output
(default)	Decimal	Serial.println(23)	23
HEX	Hexadecimal	Serial.println(23, HEX)	17
OCT	Octal	Serial.println(23, OCT)	27
BIN	Binary	Serial.println(23, BIN)	00010111

Receiving Data — Arduino Reading from Serial Monitor

The Serial Buffer

The Arduino maintains an **incoming serial buffer** that stores received bytes until the sketch reads them. `Serial.available()` reports how many bytes are waiting; `Serial.read()` retrieves them one byte at a time.

Echo Program — Arduino Reflects Incoming Data

```
char data; // Holds one incoming character

void setup() {
  Serial.begin(9600);
}

void loop() {
  if (Serial.available() > 0) { // Wait for data in buffer
    data = Serial.read();      // Read one byte
    Serial.print(data);        // Echo it back
  }
}
```

Every character typed in the Serial Monitor and sent is immediately echoed back and displayed.

Characters vs Integers in Serial Communication

All data received over serial arrives as **ASCII characters**, not raw integers. The ASCII table maps every character to a 7-bit number (0–127).

Example: Sending the character '5' over serial transmits the ASCII value **53**, not the integer 5.

Three approaches handle this:

Approach 1 — Direct character comparison (for single digits or letters):

```
if (Serial.read() == '1') { ... } // Single quotes = character literal
```

Approach 2 — Subtract ASCII zero (for single digit 0–9):

```
int val = Serial.read() - '0'; // Converts '5' (ASCII 53) to integer 5
```

Does not work for multi-digit numbers.

Approach 3 — `Serial.parseInt()` (most versatile — handles multi-digit integers):

```
int val = Serial.parseInt(); // Extracts full integer; waits for comma or newline
```

Controlling an LED Using Single Character Commands

Circuit: LED with 220Ω resistor connected to pin 9.

Program: Sends '1' to turn ON, '0' to turn OFF.

```
const int LED = 9;
char data;

void setup() {
  Serial.begin(9600);
  pinMode(LED, OUTPUT);
}
```

```

void loop() {
  if (Serial.available() > 0) {
    data = Serial.read();

    if (data == '1') {          // Character '1' received
      digitalWrite(LED, HIGH);
      Serial.println("LED ON");
    }
    else if (data == '0') {     // Character '0' received
      digitalWrite(LED, LOW);
      Serial.println("LED OFF");
    }
    // Newline '\n' from terminal: falls through — neither condition met
    // Using else if instead of else prevents newline from triggering OFF
  }
}

```

Important: The Serial Monitor appends a `\n` (newline) to each sent message. Using `else if` ensures the newline character is silently ignored rather than triggering the LED off state.

Controlling RGB LED — Sending Multiple Values via Serial

To control an RGB LED, three comma-separated percentage values (0–100) are sent: **Format:** 80,10,80 followed by Enter

Circuit: Common-anode RGB LED with cathodes on pins 11 (R), 10 (G), 9 (B) through 220Ω resistors.

```

const int RED  = 11;
const int GREEN = 10;
const int BLUE  = 9;

void setup() {
  Serial.begin(9600);
  Serial.setTimeout(10); // parseInt() timeout: 10ms

  pinMode(RED, OUTPUT);
  pinMode(GREEN, OUTPUT);
  pinMode(BLUE, OUTPUT);

  // Common-anode: HIGH = OFF
  digitalWrite(RED, HIGH);
  digitalWrite(GREEN, HIGH);
  digitalWrite(BLUE, HIGH);
}

void loop() {
  if (Serial.available() > 0) {
    int val1 = Serial.parseInt(); // Extract first integer (before first comma)
    int val2 = Serial.parseInt(); // Extract second integer (before second comma)
    int val3 = Serial.parseInt(); // Extract third integer (before newline)

    // Flush any remaining characters in the buffer
    while (Serial.available()) { Serial.read(); }
  }
}

```

```

// Constrain to 0–100%
int v1 = constrain(val1, 0, 100);
int v2 = constrain(val2, 0, 100);
int v3 = constrain(val3, 0, 100);

// Map percentage to PWM (inverted for common-anode)
int rval = map(v1, 0, 100, 255, 0); // 0% → 255 (OFF), 100% → 0 (ON)
int gval = map(v2, 0, 100, 255, 0);
int bval = map(v3, 0, 100, 255, 0);

analogWrite(RED, rval);
analogWrite(GREEN, gval);
analogWrite(BLUE, bval);

// Confirm values on Serial Monitor
Serial.println("Red: " + String(v1) + "%");
Serial.println("Green: " + String(v2) + "%");
Serial.println("Blue: " + String(v3) + "%\n");
}
}

```

How parseInt() works:

- Waits for a non-numeric character (comma or newline) to signal end of the current number.
- Extracts the digits before that delimiter and returns them as an integer.
- `setTimeout(10)` prevents the program from hanging if an incomplete message is sent — after 10 ms without new data, `parseInt()` assumes the number is complete.

Serial Communication with Processing (Desktop App)

The Arduino Serial port is accessible from **any desktop programming language** that supports serial communication — Python, Java, Node.js, C, and Processing. Processing is used here as it has syntax nearly identical to Arduino.

Arduino → Processing: Sending Potentiometer Data

Arduino code sends ADC value every 50 ms:

```

const int POT = 0;
int val;

void setup() {
  Serial.begin(9600);
}

void loop() {
  val = map(analogRead(POT), 0, 1023, 0, 255); // Map to 0–255
  Serial.println(val); // Send value to computer
  delay(50); // Avoid flooding serial buffer
}

```

Processing code reads the value and changes window color:

```

import processing.serial.*;
Serial port;
float brightness = 0;

```

```

void setup() {
  size(500, 500);
  port = new Serial(this, "COM3", 9600); // Match Arduino baud rate
  port.bufferUntil('\n');           // Trigger event on newline
}

void draw() {
  background(0, 0, brightness); // Blue channel controlled by pot
}

void serialEvent(Serial port) {
  brightness = float(port.readStringUntil('\n')); // Convert string to float
}

```

Key Processing functions:

Function	Purpose
new Serial(this, port, baud)	Opens serial port at given baud rate
port.bufferUntil('\n')	Buffer incoming data; trigger event at newline
serialEvent()	Called automatically when bufferUntil condition is met
port.readStringUntil('\n')	Reads buffered string up to newline
float()	Converts string to floating point number

Only one program can access a serial port at a time. **Close the Arduino Serial Monitor** before running a Processing sketch that uses the same port.

Processing → Arduino: Color Picker Controlling RGB LED

Processing sends comma-separated RGB values when the user clicks on a color image:

```

void mousePressed() {
  color c = get(mouseX, mouseY); // Get pixel color at click
  int r = int(map(red(c), 0, 255, 0, 100));
  int g = int(map(green(c), 0, 255, 0, 100));
  int b = int(map(blue(c), 0, 255, 0, 100));
  String colors = r + "," + g + "," + b + "\n";
  port.write(colors); // Send to Arduino
}

```

The Arduino receives this string using the `parseInt()` RGB control code from Section 1.8 and sets the LED color accordingly.

Software Serial (Overview)

The Arduino Uno has only **one hardware UART** (pins 0 and 1). When this port is used for USB communication, no additional hardware serial device can be connected on the same port. **Software Serial** solves this by emulating a serial port on any two digital pins in software.

```

#include <SoftwareSerial.h>
SoftwareSerial mySerial(RX_pin, TX_pin); // Define custom RX, TX pins

void setup() {
  Serial.begin(9600); // Hardware serial (USB/computer)
  mySerial.begin(9600); // Software serial (secondary device) }

```

Feature	Hardware Serial	Software Serial
Pins	Fixed: 0 (RX), 1 (TX)	Any two digital pins
Speed	Full speed, hardware-buffered	Slower; software overhead
Simultaneous	Cannot use for two devices	Allows secondary serial device
Use case	USB/computer communication	GPS, Bluetooth, secondary device

THE I²C BUS

Introduction to I²C

I²C (Inter-Integrated Circuit, pronounced "eye-squared-see" or "eye-two-see") is a two-wire serial communication protocol that allows a **single master device** to communicate with **multiple slave devices** over just two shared lines. It was developed by **Philips Semiconductors in the early 1980s** for low-speed communication between integrated circuits on the same board.

I²C is also generically called the **"two-wire protocol"** — any device labeled as using the "two-wire interface" is almost always I²C compatible.

Why I²C Matters

With only two wires serving all devices on a bus, I²C enables a microcontroller to interface with many peripherals (sensors, displays, EEPROMs, etc.) while consuming very few GPIO pins — a critical advantage in resource-constrained embedded systems.

Typical I²C speeds:

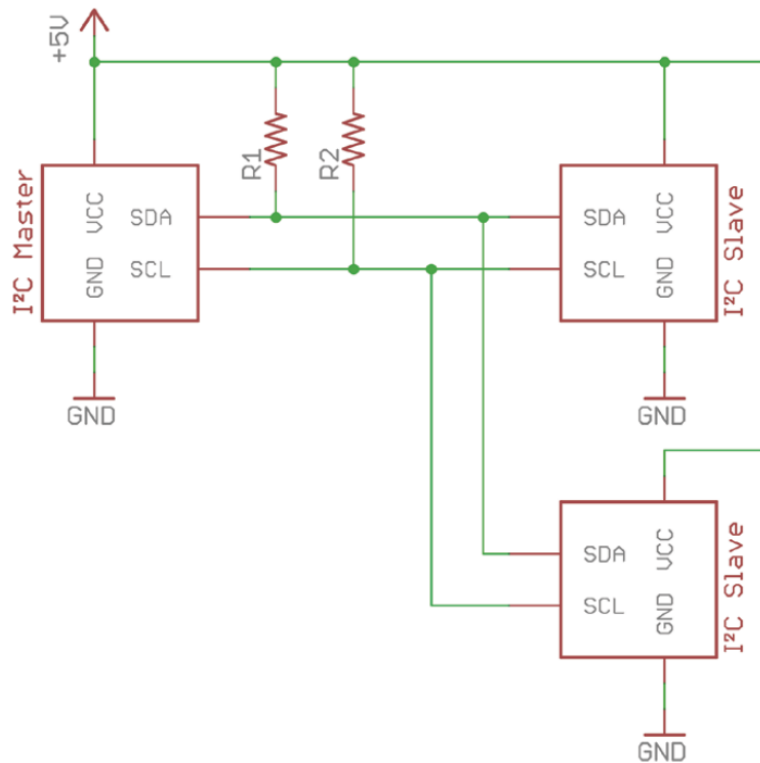
- Standard mode: 100 kbps
- Fast mode: 400 kbps
- High-speed mode: up to a few Mbps (hardware dependent)

I²C Hardware Design

The Two Communication Lines

An I²C bus uses exactly two signal lines shared among all devices:

Line	Name	Direction	Function
SDA	Serial Data	Bidirectional	Carries all data bits between master and slaves
SCL	Serial Clock	Master → All	Synchronizes data transmission timing



I²C Pins on Arduino Uno

On the Arduino Uno, the I²C hardware interface is **multiplexed** with analog pins:

Arduino Pin	I ² C Function
A4	SDA (Serial Data)
A5	SCL (Serial Clock)

When the Wire library is initialized, these pins are internally connected to the ATmega328P's hardware I²C controller. They **cannot be used as analog inputs** while I²C is active. Newer Arduino board revisions also expose dedicated SDA/SCL pins above the AREF pin — these are electrically identical to A4/A5.

Hardware Requirements — Pull-Up Resistors

Why Pull-Up Resistors Are Mandatory

The I²C bus uses **open-drain** (or open-collector) I/O pins. This means:

- Each pin has only a transistor to **pull the line LOW** (to GND).
- There is **no internal transistor to drive the line HIGH**.
- The line returns to HIGH only through the **external pull-up resistor** connected to VCC.

This design is intentional and provides two key advantages:

1. **Short-circuit protection:** Since no device can actively drive HIGH, two devices pulling the line in opposite directions (one HIGH, one LOW) cannot create a destructive short circuit. Only one LOW driver at a time can exist safely.
2. **Voltage level compatibility:** A 3.3V master can communicate with a 5V slave by connecting the pull-up to 5V. The LOW state is driven actively; the HIGH state floats up to whatever voltage the pull-up is connected to.

Choosing the Pull-Up Resistor Value

Resistor Value	Effect	Risk
Too low (strong pull-up)	Fast signal rise, high current	Open-drain transistors may not overcome the pull-up to reach LOW
Too high (weak pull-up)	Low current, power efficient	Slow signal rise time limits maximum I ² C speed
Typical: 4.7 kΩ	Good balance for most Arduino projects at standard speed	—

For most Arduino prototyping applications, **4.7 kΩ pull-up resistors on both SDA and SCL** work reliably at standard 100 kbps speeds.

Communication Scheme and ID Numbers

Master-Slave Architecture

The I²C bus operates on a strict **master-slave** model:

- **Master** (Arduino): Initiates all communication. Generates the clock signal. No slave can initiate data transfer.
- **Slave** (sensor, display, etc.): Responds only when addressed by the master.

Since all devices share the same two wires, **every message reaches every slave simultaneously**. To ensure only the intended device responds, each slave has a **unique 7-bit address**.

7-bit Slave Address

Every I²C device has a unique address from 0 to 127 (7 bits = 128 possible addresses). When the master initiates communication, it first broadcasts the **target slave address**. Only the slave with that address responds; all others ignore the message.

Address types:

- **Fixed address:** Permanently set during manufacturing (e.g. TC74A0 has address 1001000 = 72 decimal).
- **Selectable address:** The device has address-select (AS) pins that can be tied to VCC, GND, or left floating — each configuration gives a different address. This allows multiple identical chips on one bus.

Example — TC74 Temperature Sensor Address Options:

Part Number	I ² C Address (Binary)	Decimal
TC74A0-5.0VAT	1001000	72
TC74A1-5.0VAT	1001001	73
TC74A2-5.0VAT	1001010	74
...	...	...

Up to 8 TC74 sensors can coexist on one I²C bus, each with a different address.

I²C Communication Sequence — Step by Step

Every I²C transaction follows this sequence:

1. Master sends a **START bit** — signals the beginning of communication to all slaves.
2. Master sends the **7-bit slave address** of the target device.
3. Master sends a **Read (1) or Write (0) bit** — specifies the direction of data transfer.
4. Addressed slave sends an **ACK (Acknowledge) bit** — a logic LOW indicating it received the address.
5. Data transfer begins:
 - **Write mode:** Master sends 1 byte; slave responds with ACK after each byte.
 - **Read mode:** Slave sends 1 byte; master sends ACK after each byte.
6. Master sends a **STOP bit** — signals end of communication; bus released.

Write Byte Format

S	Address	WR	ACK	Command	ACK	Data	ACK	P
	7 Bits			8 Bits		8 Bits		

Slave Address

Command Byte: selects which register you are writing to.

Data Byte: data goes into the register set by the command byte.

Read Byte Format

S	Address	WR	ACK	Command	ACK	S	Address	RD	ACK	Data	NACK	P
	7 Bits			8 Bits			7 Bits			8 Bits		

Slave Address

Command Byte: selects which register you are reading from.

Slave Address: repeated due to change in data-flow direction.

Data Byte: reads from the register set by the command byte.

Receive Byte Format

S	Address	RD	ACK	Data	NACK	P
	7 Bits			8 Bits		

S = START Condition

P = STOP Condition

Shaded = Slave Transmission

Data Byte: reads data from the register commanded by the last Read Byte or Write Byte transmission.

The Arduino Wire Library

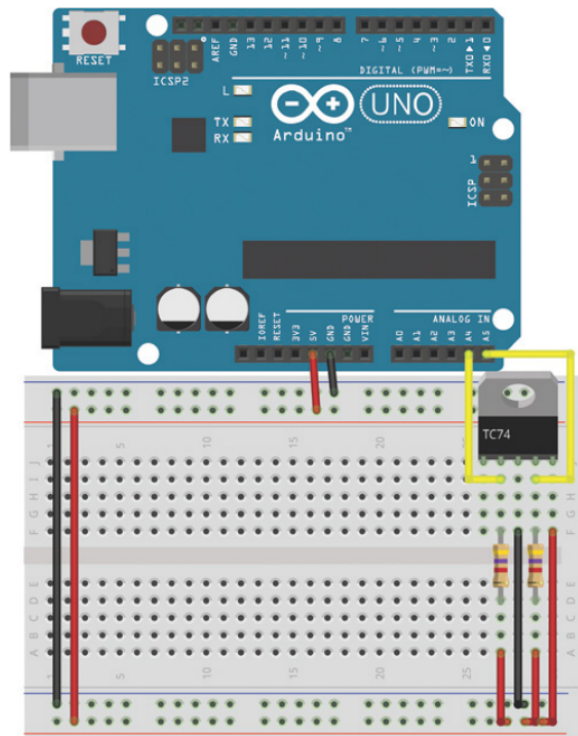
Arduino's I²C communication is handled through the built-in **Wire library**. It abstracts all the low-level timing, start/stop bits, and ACK handling.

Wire Library — Key Functions

Function	Purpose
Wire.begin()	Initializes I ² C as master; activates A4/A5 pins for I ² C
Wire.beginTransmission(address)	Starts communication with slave at given address
Wire.write(value)	Sends one byte to the slave (used after beginTransmission)
Wire.endTransmission()	Sends STOP bit; completes the write transaction
Wire.requestFrom(address, count)	Requests count bytes from slave; returns number of bytes received
Wire.read()	Reads one byte from the receive buffer (after requestFrom)

Communicating with an I²C Temperature Sensor — TC74

Hardware Setup



The TC74 is a digital I²C temperature sensor that outputs temperature **directly in degrees Celsius** as an 8-bit integer — no analog conversion or formula needed.

Understanding the TC74 Datasheet

The TC74 has two internal registers:

Register	Contents	Access
Temperature (0x00)	Current temperature in °C as 8-bit signed integer	Read
Configuration	Standby mode, data-ready status	Read/Write

Read Byte Procedure (from datasheet):

1. Send slave address in **write mode** + write 0 → selects the temperature data register.
2. Send slave address in **read mode** + request **1 byte** → receive temperature.
3. Confirm 1 byte was received.

Temperature Reading Program

```
#include <Wire.h>    // Include I2C library

int temp_address = 72; // TC74A0 address: 1001000 binary = 72 decimal

void setup() {
  Serial.begin(9600); // Initialize serial for output
  Wire.begin();       // Initialize I2C as master (activates A4, A5)
}
```

```

void loop() {
  // STEP 1: Write mode — select temperature data register (register 0)
  Wire.beginTransmission(temp_address); // Start with slave address
  Wire.write(0);                        // Request register 0 (temp data)
  Wire.endTransmission();              // Send STOP bit

  // STEP 2: Read mode — request 1 byte from the sensor
  int returned_bytes = Wire.requestFrom(temp_address, 1);

  // STEP 3: Error check — if 0 bytes returned, sensor not connected
  if (returned_bytes == 0) {
    Serial.println("I2C Error"); // Print error message
    while(1);                    // Halt execution (endless loop)
  }

  // STEP 4: Read temperature value (8-bit, degrees Celsius)
  int c = Wire.read();

  // STEP 5: Convert Celsius to Fahrenheit
  int f = round(c * 9.0/5.0 + 32.0);

  // STEP 6: Print both values
  Serial.print(c);
  Serial.print("C ");
  Serial.print(f);
  Serial.println("F");

  delay(500); // Read every 500ms
}

```

Line-by-line explanation of Wire functions:

Code Line	What It Does
Wire.begin()	Initializes I ² C hardware; sets A4/A5 as SDA/SCL
Wire.beginTransmission(72)	Sends START bit + address 72 + WRITE bit
Wire.write(0)	Sends 0 → tells TC74 to point to temperature register
Wire.endTransmission()	Sends STOP bit — ends write phase
Wire.requestFrom(72, 1)	Sends START + address 72 + READ bit; requests 1 byte; returns count received
Wire.read()	Reads the 1 byte of temperature data from buffer

Celsius to Fahrenheit Conversion

$$F = (C \times 9.0/5.0) + 32.0$$

Using 9.0 and 5.0 (floating point literals) ensures floating-point division. round() converts the result to the nearest integer.

I²C Temperature Monitoring System — Extended Application

Combining I²C temperature reading with shift register LED display and Processing visualization creates a complete temperature monitoring system.

Modified Serial Output Format for Processing Parsing

Instead of spaces and newlines, use commas and periods as delimiters:

```
Serial.print(c);
Serial.print("C,");
Serial.print(f);
Serial.print("F."); // Period signals end of complete message
Error format (same structure for consistent parsing):
cpp
Serial.print("Err,Err.");
Processing Sketch — Temperature Display
java
import processing.serial.*;
Serial port;
String temp_c = "";
String temp_f = "";
String data = "";
int index = 0;
PFont font;

void setup() {
  size(400, 400);
  port = new Serial(this, "COM9", 9600);
  port.bufferUntil('.'); // Buffer until period — end of one reading

  // Load pre-generated font (created via Tools → Create Font in Processing)
  font = loadFont("AgencyFB-Bold-200.vlw");
  textFont(font, 200);
}

void draw() {
  background(0, 0, 0); // Black background
  fill(46, 209, 2); // Green for Celsius
  text(temp_c, 70, 175);
  fill(0, 102, 153); // Blue for Fahrenheit
  text(temp_f, 70, 370);
}

void serialEvent(Serial port) {
  data = port.readStringUntil('.'); // Read up to period
  data = data.substring(0, data.length() - 1); // Remove trailing period
  index = data.indexOf(","); // Find comma position
  temp_c = data.substring(0, index); // Extract Celsius part
  temp_f = data.substring(index + 1, data.length()); // Extract Fahrenheit part
}
```

Data flow: Arduino reads TC74 → formats as "24C,75F." → sends over serial → Processing reads until .
→ splits at , → displays on screen.

THE SPI BUS

The **Serial Peripheral Interface (SPI)** bus (pronounced "spy") is a **full-duplex, synchronous** serial communication protocol originally developed by Motorola. It enables simultaneous bidirectional data transfer between a master device and one or more slave devices at high speeds.

Unlike I²C, SPI uses **separate lines for sending and receiving data** and a dedicated **slave select line per device**, eliminating the need for device addressing. This makes SPI generally easier to implement and capable of operating at higher speeds than I²C.

Key Characteristics

Feature	Detail
Communication type	Full-duplex (simultaneous send and receive)
Synchronization	Synchronous — data clocked by shared SCLK signal
Lines required	3 shared + 1 per slave (SS)
Addressing	Not needed — SS pin selects the slave
Speed	Faster than I ² C; depends on slave device capability
Pull-up resistors	Not required (push-pull I/O pins)
Arduino library	SPI.h (built-in)

SPI Hardware Configuration

The Four SPI Lines

An SPI system uses the following signal lines:

Line	Full Name	Direction	Function
MOSI	Master Out / Slave In	Master → Slave	Master sends data to slave
MISO	Master In / Slave Out	Slave → Master	Slave sends data to master
SCLK	Serial Clock	Master → All	Synchronizes data on each clock edge
SS	Slave Select (Chip Select)	Master → Each Slave	Pulled LOW to activate a specific slave

Total pins required on master = 3 + n, where n is the number of slave devices (one SS pin per slave).

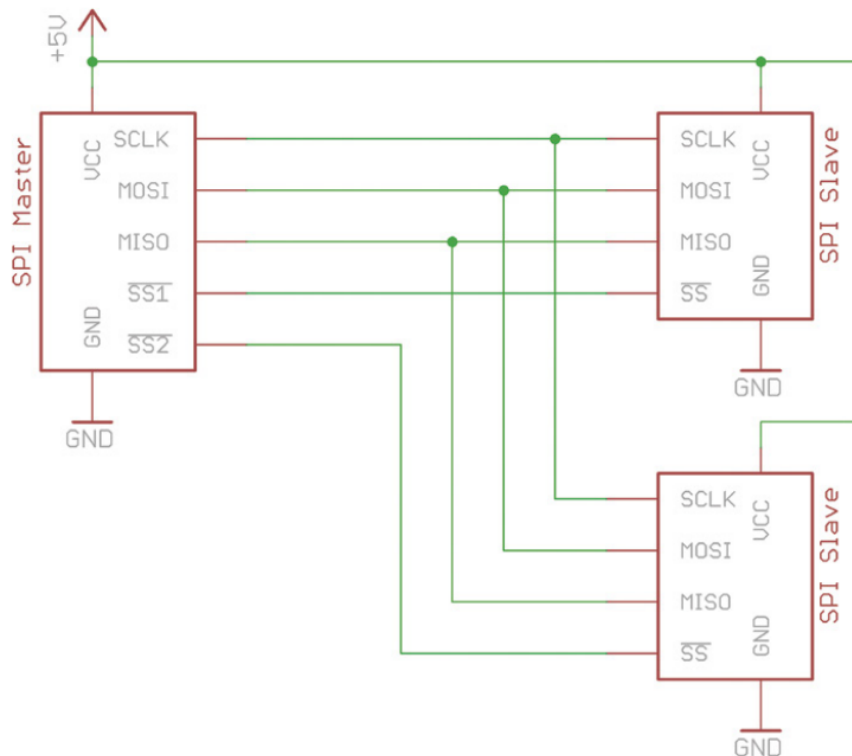
Alternative naming conventions (SPI is not a universal standard — manufacturers use different names):

Standard Name	Alternative Names
SS	CS (Chip Select), /CS
SCLK	CLK, SCK
MOSI	SDI (Serial Data In) — on slave side
MISO	SDO (Serial Data Out) — on slave side

Arduino Uno SPI Pin Mapping

Arduino Uno Pin	SPI Function
10	SS (Slave Select / Chip Select)
11	MOSI (Master Out / Slave In)
12	MISO (Master In / Slave Out)
13	SCLK (Serial Clock)

These are hardware SPI pins multiplexed on the ATmega328P. Using the hardware SPI interface is always preferred over software emulation — the hardware buffers data efficiently and frees CPU cycles.



SPI Communication Modes

SPI data can be clocked on different clock edges depending on the device. This is defined by two parameters:

- **Clock Polarity (CPOL):** Default state of SCLK when idle — LOW (0) or HIGH (1)
- **Clock Phase (CPHA):** Which clock edge captures data — Rising or Falling

These combine to give **four SPI modes**:

SPI Mode	Clock Polarity	Clock Phase
Mode 0	Low at Idle	Data Capture on Clock Rising Edge
Mode 1	Low at Idle	Data Capture on Clock Falling Edge
Mode 2	High at Idle	Data Capture on Clock Falling Edge
Mode 3	High at Idle	Data Capture on Clock Rising Edge

When using a third-party library designed for a specific device, the library automatically configures the correct SPI mode. Understanding modes helps in troubleshooting and when writing low-level SPI code.

SPI Communication Scheme

Basic Communication Process

The SPI master controls the entire transaction:

1. Pull the target slave's **SS pin LOW** — activates that slave.
2. Toggle the **SCLK** clock line up and down at the agreed-upon speed.
3. For each clock cycle:
 - Send **1 bit on MOSI** (master to slave)
 - Receive **1 bit on MISO** (slave to master) — simultaneously
4. Continue until transmission is complete.
5. Return the **SS pin HIGH** — deactivates the slave.

Since all slaves receive MOSI, MISO, and SCLK simultaneously, only the slave with SS pulled LOW processes and responds to the data. All others ignore the bus.

Critical rule: Only one SS pin should be LOW at any given time. Activating two slaves simultaneously causes data collision on the shared MISO line.

Comparing SPI to I²C and UART

This is a frequently examined topic. The three main serial protocols used with Arduino are compared below:

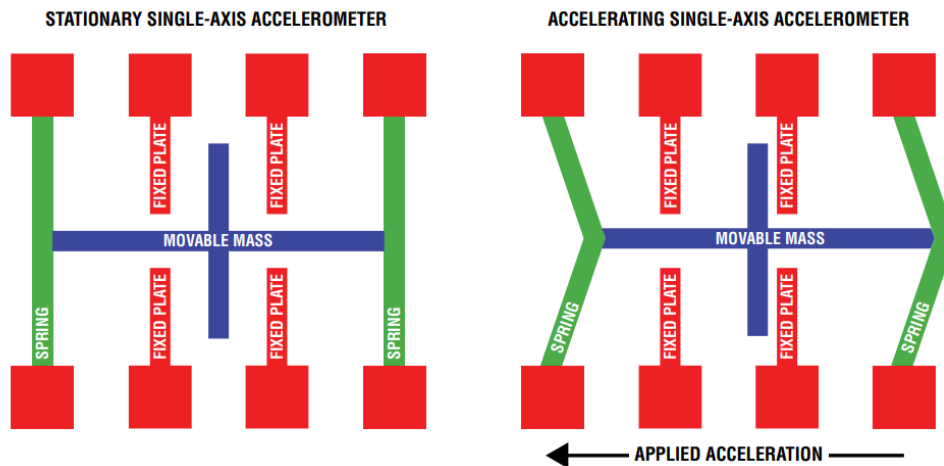
Feature	SPI	I ² C	UART
Communication type	Full-duplex	Half-duplex	Full-duplex
Lines required	3 + 1 per slave	2 (shared)	2 (TX, RX)
Clock	Master-generated (SCLK)	Master-generated (SCL)	No clock — baud rate agreed in advance
Speed	Fastest	Moderate	Depends on baud rate
Addressing	No — uses SS pin	Yes — 7-bit address	No — point-to-point
Multiple slaves	Yes (one SS per slave)	Yes (up to 128 addresses)	Difficult — not designed for it
Pull-up resistors	Not needed	Required on SDA and SCL	Not needed
Ease of use	Easier than I ² C	More complex	Simplest
Voltage flexibility	Limited	Better (open-drain allows mixed voltages)	Requires matching levels
Arduino hardware support	Yes (SPI.h)	Yes (Wire.h)	Yes (Serial)
Common use	High-speed sensors, displays, SD cards	Multiple slow sensors	Debug, GPS, Bluetooth

What Is an Accelerometer?

An **accelerometer** is a sensor that measures **linear acceleration** along one or more axes. It is one of the three core positioning sensors found in modern smartphones (along with a gyroscope for rotational motion and a magnetometer for magnetic fields).

Physical Working Principle (MEMS Accelerometer)

Modern accelerometers are **MEMS** (Micro Electro Mechanical Systems) devices — microscopic mechanical structures etched onto a silicon chip.



Right (accelerating): The same structure but the movable mass has shifted toward one side — show the gaps between the movable mass and fixed plates are unequal. Label: APPLIED ACCELERATION with arrow.) The movable mass and fixed plates form tiny capacitors. When the device accelerates, the mass shifts due to inertia (like being pushed into a car seat when accelerating). This changes the gap between the movable mass and fixed plates, changing the capacitance. The electronics measure this capacitance change and convert it to an acceleration value.

Gravity and Orientation Detection

Since gravity is a constant 9.8 m/s^2 acceleration toward Earth's center, the accelerometer always detects gravity on the axis pointing toward Earth:

- **Upright:** Z-axis reads approximately **+1G**
- **Upside down:** Z-axis reads approximately **-1G**
- **Freefall:** All axes read approximately **0G** (gravity not felt)

This makes accelerometers excellent for **orientation detection** without any external reference.

The LIS3DH SPI Accelerometer

The **STMicroelectronics LIS3DH** is a 3-axis digital accelerometer that supports both SPI and I²C interfaces. It measures acceleration along the X, Y, and Z axes simultaneously.

The LIS3DH is only available in an **SMT (Surface Mount Technology)** package — too small for a breadboard. The **Adafruit LIS3DH breakout board** adapts this chip to a breadboard-friendly form factor, includes an onboard voltage regulator and level shifter for 5V Arduino compatibility.

Sensitivity Setting

The LIS3DH returns a **signed 16-bit integer** (-32768 to +32767) for each axis. The sensitivity range is selectable:

Setting	Range	Meaning of +32767
LIS3DH_RANGE_2_G	±2G	+32767 = +2G
LIS3DH_RANGE_4_G	±4G	+32767 = +4G
LIS3DH_RANGE_8_G	±8G	+32767 = +8G

To convert raw reading to actual G value:

Actual G = (raw value / 32768.0) × full-scale range

For ±2G range: **Actual G = (raw / 32768.0) × 2.0**

Installing Third-Party Libraries

Many sophisticated devices like the LIS3DH have **community-developed third-party libraries** that abstract complex register-level SPI communication into simple function calls.

Installing via Arduino Library Manager

1. Open Arduino IDE → **Sketch** → **Include Library** → **Manage Libraries**
2. Search for "**Adafruit Unified Sensor**" → Install
3. Search for "**Adafruit LIS3DH**" → Install

The **Adafruit LIS3DH library** depends on the **Adafruit Unified Sensor library** — both must be installed. The LIS3DH library itself includes and uses the Arduino SPI library internally; no separate SPI #include is needed.

After installation, example sketches are accessible via **File** → **Examples** → **Adafruit LIS3DH**.

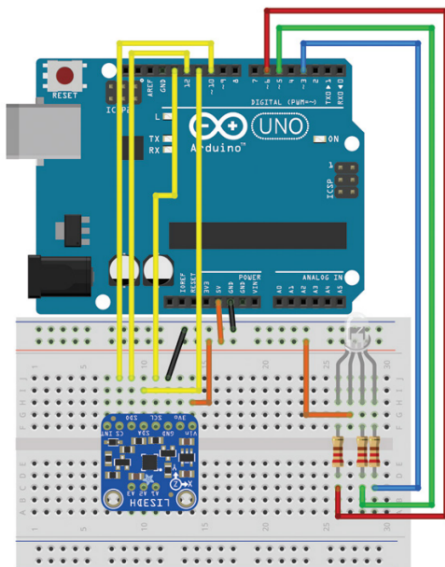
SPI Wiring — LIS3DH Accelerometer to Arduino Uno

Connect the Adafruit LIS3DH breakout board to the Arduino Uno as follows:

Breakout Board Pin	Arduino Uno Pin	Function
VIN	5V	Power supply
GND	GND	Ground
SCK	13	Serial Clock (SCLK)
SDI (MOSI)	11	Master Out / Slave In
SDO (MISO)	12	Master In / Slave Out
CS	10	Chip Select (SS)

Additionally, connect a **common-anode RGB LED** through 220Ω resistors:

Arduino Pin	LED Color
6	Red cathode
5	Green cathode
3	Blue cathode
5V	Common anode



Program 1 — Orientation Sensor

The LIS3DH Z-axis always experiences gravity. When the Arduino is upright, Z reads positive (+1G). When upside down, Z reads negative (−1G). This drives a green/red LED indicator.

```
#include <Adafruit_LIS3DH.h> // Includes SPI and Unified Sensor internally

const int RED_PIN = 6; // Red LED cathode → pin 6
const int GREEN_PIN = 5; // Green LED cathode → pin 5
const int CS_PIN = 10; // Chip Select → pin 10

// Create accelerometer object using hardware SPI (pins 11,12,13 by default)
Adafruit_LIS3DH accel = Adafruit_LIS3DH(CS_PIN);

void setup() {
  Serial.begin(9600);

  // Initialize accelerometer — returns false if not found
  if (!accel.begin()) {
    Serial.println("Could not find accelerometer.");
    while (1); // Halt — check wiring if stuck here
  }

  // Set sensitivity to ±2G range (highest resolution)
  accel.setRange(LIS3DH_RANGE_2_G);

  // Set LED cathode pins as outputs — HIGH = OFF (common anode)
  pinMode(RED_PIN, OUTPUT);
  digitalWrite(RED_PIN, HIGH); // Start with red OFF
  pinMode(GREEN_PIN, OUTPUT);
  digitalWrite(GREEN_PIN, HIGH); // Start with green OFF
}

void loop() {
  accel.read(); // Fetch latest X, Y, Z acceleration values

  // Print raw Z value and computed G value
  Serial.print("Raw: ");
  Serial.print(accel.z);
  Serial.print("\tActual: ");
  Serial.print((float(accel.z) / 32768.0) * 2.0);
  Serial.println("G");

  // Orientation detection using Z-axis sign
  if (accel.z < 0) { // Upside down: Z is negative
    digitalWrite(RED_PIN, LOW); // Red ON (cathode LOW = ON)
    digitalWrite(GREEN_PIN, HIGH); // Green OFF
  } else { // Upright: Z is positive
    digitalWrite(RED_PIN, HIGH); // Red OFF
    digitalWrite(GREEN_PIN, LOW); // Green ON (cathode LOW = ON)
  }

  delay(100); // Sample every 100ms
}
```

Key library functions used:

Function	Purpose
Adafruit_LIS3DH(CS_PIN)	Creates accelerometer object; CS_PIN specifies SS pin; other SPI pins default to hardware pins 11,12,13
accel.begin()	Initializes SPI communication with LIS3DH; returns false if device not found
accel.setRange(LIS3DH_RANGE_2_G)	Sets measurement sensitivity to $\pm 2\text{G}$
accel.read()	Reads current X, Y, Z acceleration into accel.x, accel.y, accel.z
accel.z	Raw 16-bit signed integer Z-axis reading (-32768 to $+32767$)

Initialization error handling:

```
if (!accel.begin()) { while(1); }
```

begin() returns a Boolean — true if initialization succeeded, false if not. The ! negates it: if initialization **failed**, print error and halt with while(1).

Program 2 — Audiovisual Instrument (3-Axis)

All three accelerometer axes independently trigger musical notes and control LED colors.

Gravity Normalization

The Z-axis always experiences $+1\text{G}$ from gravity when the device is held face-up. To detect only **actual motion** (not constant gravity), subtract the raw equivalent of 1G :

With $\pm 2\text{G}$ range: $+32767$ represents $+2\text{G}$, so $+1\text{G} = 32767/2 \approx 16384$ raw units

```
long norm_z = accel.z - 16384; // Remove gravity from Z reading
```

Now norm_z reads ≈ 0 when stationary — only deviates when the device actually accelerates in Z.
Complete Audiovisual Instrument Program

```
#include <Adafruit_LIS3DH.h>
```

```
const int RED_PIN = 6;
const int GREEN_PIN = 5;
const int BLUE_PIN = 3;
const int SPEAKER = 9; // Piezo buzzer with 220Ω series resistor
const int CS_PIN = 10;
```

```
// Pentatonic scale notes (C D E G A C2) — always sound harmonious together
#define NOTE_C 262 // Hz
#define NOTE_D 294
#define NOTE_E 330
#define NOTE_G 392
#define NOTE_A 440
#define NOTE_C2 523
```

```
Adafruit_LIS3DH accel = Adafruit_LIS3DH(CS_PIN);
```

```
void setup() {
  Serial.begin(9600);
```

```

if (!accel.begin()) {
  Serial.println("Could not find accelerometer.");
  while (1);
}

accel.setRange(LIS3DH_RANGE_2_G);

// Set all LED cathode pins — HIGH = OFF (common anode)
pinMode(RED_PIN, OUTPUT); digitalWrite(RED_PIN, HIGH);
pinMode(GREEN_PIN, OUTPUT); digitalWrite(GREEN_PIN, HIGH);
pinMode(BLUE_PIN, OUTPUT); digitalWrite(BLUE_PIN, HIGH);
}

void loop() {
  accel.read(); // Read all 3 axes

  // Normalize Z-axis to remove constant gravity effect
  long norm_z = accel.z - 16384;

  // Print all axis values for threshold calibration
  Serial.print(accel.x);
  Serial.print(" ");
  Serial.print(accel.y);
  Serial.print(" ");
  Serial.println(norm_z);

  // Trigger a 100ms tone when acceleration exceeds  $\pm 5000$  raw units on each axis
  if (accel.x < -5000) tone(SPEAKER, NOTE_C, 100); // X negative → C
  if (accel.x > 5000) tone(SPEAKER, NOTE_D, 100); // X positive → D
  if (accel.y < -5000) tone(SPEAKER, NOTE_E, 100); // Y negative → E
  if (accel.y > 5000) tone(SPEAKER, NOTE_G, 100); // Y positive → G
  if (norm_z < -5000) tone(SPEAKER, NOTE_A, 100); // Z negative → A
  if (norm_z > 5000) tone(SPEAKER, NOTE_C2, 100); // Z positive → C2

  // Map acceleration magnitude to LED brightness (common anode: 255=OFF, 0=ON)
  // abs() gives magnitude regardless of direction
  // map: raw 5000–20000 → PWM 255–0 (inverted for common anode)
  analogWrite(RED_PIN,
    constrain(map(abs(accel.x), 5000, 20000, 255, 0), 0, 255));
  analogWrite(GREEN_PIN,
    constrain(map(abs(accel.y), 5000, 20000, 255, 0), 0, 255));
  analogWrite(BLUE_PIN,
    constrain(map(abs(norm_z), 5000, 20000, 255, 0), 0, 255));
}

```

Key concepts used:

Code Element	Purpose
<code>norm_z = accel.z - 16384</code>	Removes +1G gravity bias from Z-axis
<code>tone(pin, freq, duration)</code>	Generates square wave at freq Hz for duration ms on pin
<code>abs(accel.x)</code>	Absolute value — magnitude of acceleration regardless of direction
<code>map(abs(accel.x), 5000, 20000, 255, 0)</code>	Maps motion intensity to inverted LED brightness (common anode)
<code>constrain(..., 0, 255)</code>	Clamps output to valid PWM range
Pentatonic scale	Notes C, D, E, G, A — naturally harmonious regardless of play order

MODEL QUESTIONS

1. Mention two uses of the Arduino serial port. (3 Marks)
2. Draw the SPI hardware configuration and explain the function of the communication lines. (3 Marks)
3. Explain any four functions in the Serial library. (4 Marks)
4. Explain the I2C bus protocol with a diagram. (5 Marks)
5. Write code that will turn on/off the onboard LED when '1'/'0' is sent from serial monitor. (4 Marks)
6. With a diagram, explain the working principle of a MEMS accelerometer. (5 Marks)